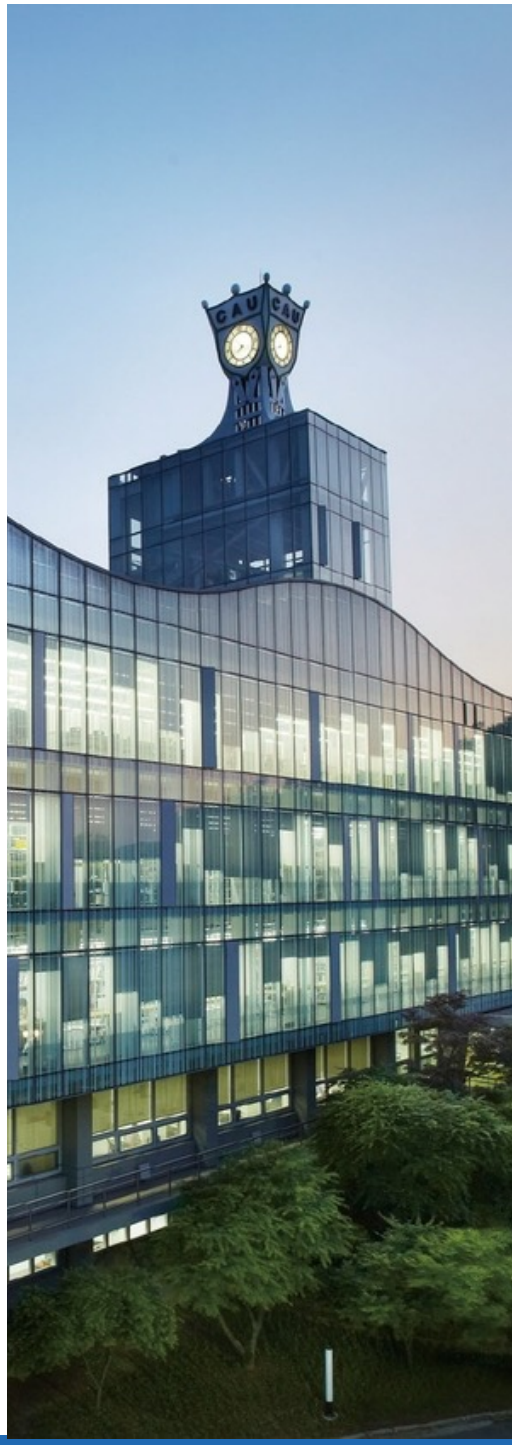
Principles of Programming Languages

Lecture 4
Lexical and Syntax Analysis
(어휘 분석과 구문 분석)

Jinsung Kim, Ph.D.
High-Performance Computing (HPC) Lab
Computer Science and Engineering
Chung-Ang University



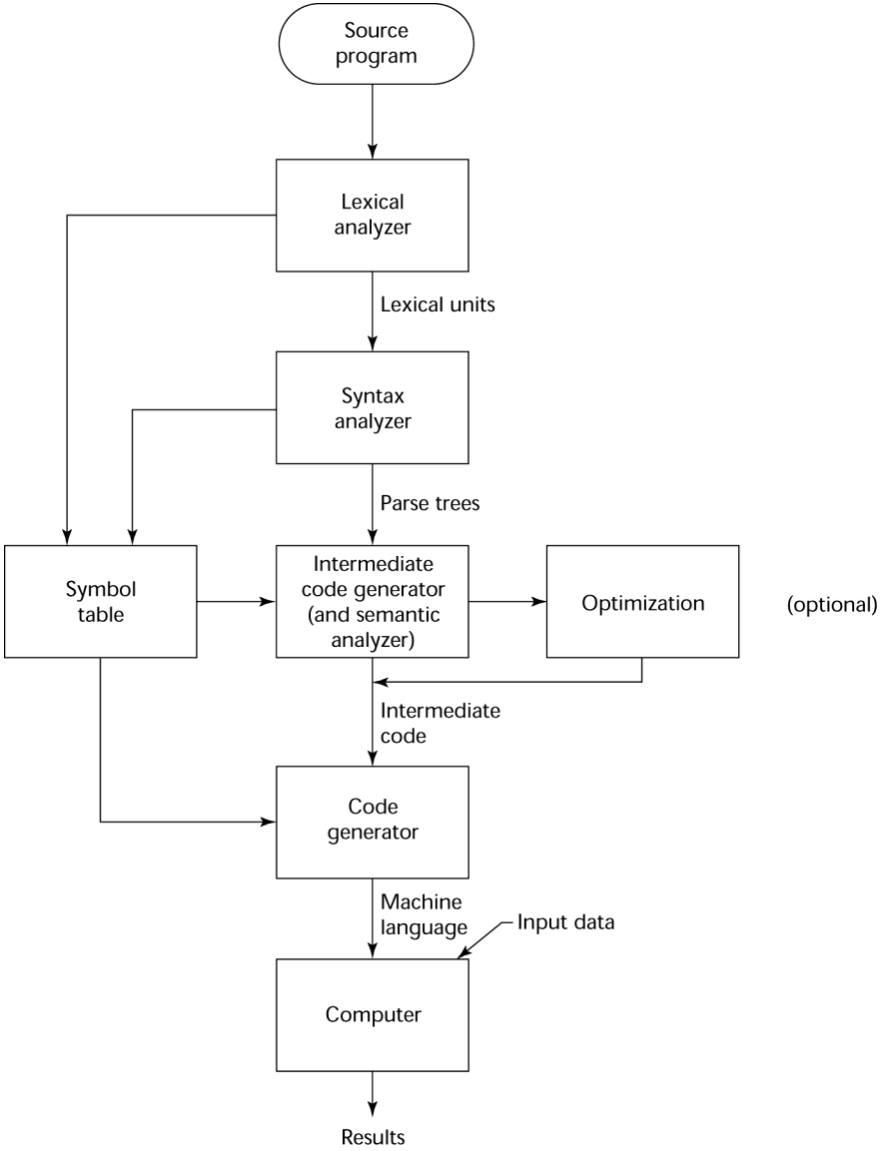
Topics

- Introduction
- Lexical Analysis
- The Parsing Problem
- Recursive-Descent Parsing
- Bottom-Up Parsing

Introduction

- Language implementation systems must analyze source code, regardless of the specific implementation approach
- Nearly all syntax analysis is based on a formal description of the syntax of the source language (BNF)

The Compliation Process (Recap)



Introduction

- **Advantages of Using BNF to Describe Syntax**
  - Provides a clear and concise syntax description
  - The parser can be based directly on the BNF
  - Parsers based on BNF are easy to maintain (modularity)

Introduction

- **Syntax Analysis**
  - The syntax analysis portion of a language processor nearly always consists of two parts:
    - A low-level part called a lexical analyzer (mathematically, a finite automaton based on a regular grammar)
    - A high-level part called a syntax analyzer, or parser (mathematically, a push-down automaton based on a context-free grammar, or BNF)

Introduction

- **Reasons to Separate Lexical and Syntax Analysis**
  - **Simplicity (단순성)**
    - Less complex approaches can be used for lexical analysis; separating them simplifies the parser
  - **Efficiency (효율성)**
    - Separation allows optimization of the lexical analyzer
  - **Portability (이식성)**
    - Parts of the lexical analyzer may not be portable, but the parser always is portable

Lexical Analysis

- **A lexical analyzer is a "pattern matcher" for character strings**
- **A lexical analyzer is a "front-end" for the parser**
- **Identifies substrings of the source program that belong together - lexemes**
  - Lexemes match a character pattern, which is associated with a lexical category called a token
  - `sum` is a lexeme; its token may be `IDENT`

Lexical Analysis

- **Example**

```
result = oldsum - value / 100;
```

Token	*Lexeme*
IDENT	result
ASSIGN_OP	=
IDENT	oldsum
SUB_OP	-
IDENT	value
DIV_OP	/
INT_LIT	100
SEMICOLON	;

Lexical Analysis (어휘분석)

- **The lexical analyzer is usually a function that is called by the parser when it needs the next token**
- **Three approaches to building a lexical analyzer:**
  - Write a formal description of the tokens and use a software tool that constructs a table-driven lexical analyzer from such a description
  - Design a state diagram that describes the tokens and write a program that implements the state diagram
  - Design a state diagram that describes the tokens and hand-construct a table-driven implementation of the state diagram

Lexical Analysis

- **The lexical analyzer is usually a function that is called by the parser when it needs the next token**
- **Three approaches to building a lexical analyzer:**
  - Write **a formal description of the tokens** and use a software tool that constructs a table-driven lexical analyzer from such a description
  - Design **a state diagram** that describes the tokens and write a program that implements the state diagram
  - Design **a state diagram** that describes the tokens and hand-construct a **table-driven** implementation of the state diagram

Lexical Analysis

- **State Diagram Design**
  - A naïve state diagram would have a transition from every state on **every character** in the source language - such a diagram would be very large!

Lexical Analysis

- **In many cases, transitions can be combined to simplify the state diagram**
  - When recognizing an identifier, all uppercase and lowercase letters are equivalent- `LETTER`
- **Use a character class that includes all letters**
  - When recognizing an integer literal, all digits are equivalent - use a `DIGIT` class

Lexical Analysis

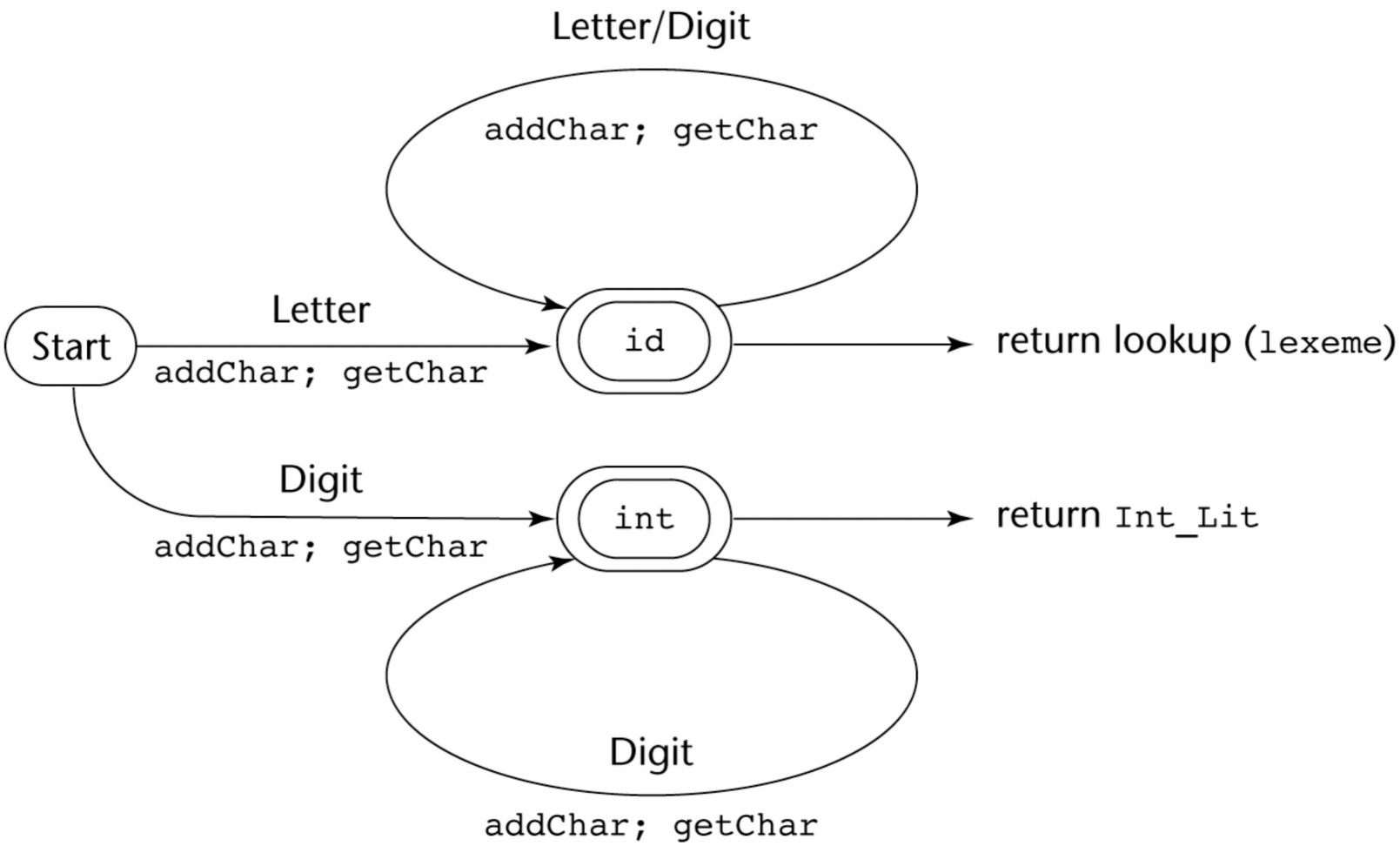
- **Reserved words and identifiers can be recognized together (rather than having a part of the diagram for each reserved word)**
  - Use a table lookup to determine whether a possible identifier is in fact a reserved word

Lexical Analysis

- **Convenient utility subprograms:**
  - `getChar`: gets the next character of input, puts it in `nextChar`, determines its class and puts the class in `charClass`
  - `addChar`: puts the character from `nextChar` into the place the lexeme is being accumulated, `lexeme`
  - `lookup`: determines whether the string in `lexeme` is a reserved word (returns a code)

Lexical Analysis

- **State Diagram**



Lexical Analysis

- **front.c (pp. 190-194)**

```
/* front.c - a lexical analyzer system for simple
             arithmetic expressions */

#include <stdio.h>
#include <ctype.h>

/* Global declarations */
/* Variables */
int  charClass;
char  lexeme [100];
char  nextChar;
int  lexLen;
int  token;
int  nextToken;
FILE *in_fp, *fopen();
```

Lexical Analysis

- front.c (pp. 190-194)

```
/* Function declarations */
void  addChar();
void  getChar();
void  getNonBlank();
int  lex();

/* Character classes */
#define LETTER 0
#define DIGIT 1
#define UNKNOWN 99

/* Token codes */
#define INT_LIT 10
#define IDENT 11
#define ASSIGN_OP 20
#define ADD_OP 21
#define SUB_OP 22
#define MULT_OP 23
#define DIV_OP 24
#define LEFT_PAREN 25
#define RIGHT_PAREN 26
```

Lexical Analysis

- front.c (pp. 190-194)

```
/*****************************************************/
/* main driver */
main() {

/* Open the input data file and process its contents */
  if ((in_fp = fopen("front.in", "r")) == NULL)
    printf("ERROR - cannot open front.in \n");
  else {
    getChar();
    do {
      lex();
    } while (nextToken != EOF);
  }
}
```

Lexical Analysis

- front.c (pp. 190-194)

```
/*****************************************************/
/* getChar - a function to get the next character of  */
             input and determine its character class */

 void getChar() {
   if  ((nextChar = getc(in_fp)) = EOF) {
     if  (isalpha(nextChar))
      charClass = LETTER;
     else if  (isdigit(nextChar))
           charClass = DIGIT;
      else  charClass = UNKNOWN;
   }
   else
     charClass = EOF;
}
```

Lexical Analysis

- front.c (pp. 190-194)

```
/*****************************************************/
/* addChar - a function to add nextChar to lexeme */
void  addChar() {
  if  (lexLen <= 98) {
    lexeme[lexLen++] = nextChar;
    lexeme[lexLen] = 0;
  }
  else
    printf("Error - lexeme is too long \n");
}

/*****************************************************/
/* getNonBlank - a function to call getChar until it
                 returns a non-whitespace character */
void getNonBlank() {
  while  (isspace(nextChar))
    getChar();
}
/
```

Lexical Analysis

- front.c (pp. 190-194)

```
*****************************************************/
/* lex - a simple lexical analyzer for arithmetic
         expressions */
int  lex() {
  lexLen = 0;
  getNonBlank();
  switch  (charClass) {

  /* Parse identifiers */
      case  LETTER:
        addChar();
        getChar();
        while  (charClass == LETTER || charClass == DIGIT) {
          addChar();
          getChar();
        }
      nextToken = IDENT;
      break;
```

Lexical Analysis

- front.c (pp. 190-194)

```
/* Parse integer literals */
    case  DIGIT:
      addChar();
      getChar();
      while  (charClass == DIGIT) {
       addChar();
       getChar();
   }
   nextToken = INT_LIT;
   break;

/* Parentheses and operators */
    case  UNKNOWN:
      lookup(nextChar);
      getChar();
      break;
```

Lexical Analysis

- front.c (pp. 190-194)

```
/* EOF */
    case  EOF:
      nextToken = EOF;
      lexeme[0] = 'E';
      lexeme[1] = 'O';
      lexeme[2] = 'F';
      lexeme[3] = 0;
      break;
 } /* End of switch */
 printf("Next token is: %d, Next lexeme is %s\n",
         nextToken, lexeme);
  return  nextToken;
} /* End of function lex */
```

Lexical Analysis

- front.c (pp. 190-194)

```
/*********************************************************/
/* lookup - a function to lookup operators and parentheses
            and return the token */

int  lookup(char  ch) {
  switch  (ch) {
    case  '(':
      addChar();
      nextToken = LEFT_PAREN;
      break;

    case  ')':
      addChar();
      nextToken = RIGHT_PAREN;
      break;
```

Lexical Analysis

- front.c (pp. 190-194)

```
case '+':
  addChar();
  nextToken = ADD_OP;
  break;

case '-':
  addChar();
  nextToken = SUB_OP;
  break;

case '*':
  addChar();
  nextToken = MULT_OP;
  break;

case '/':
  addChar();
  nextToken = DIV_OP;
  break;
```

Lexical Analysis

- front.c (pp. 190-194)

```
        default:
          addChar();
          nextToken = EOF;
          break;
      }
      return  nextToken;
    }
```

Lexical Analysis

- **Implementation:**
  : `front.c` **(pp. 190-194)**
- **Following is the output of the lexical analyzer of** `front.c` **when used on** `(sum + 47)/total`

```
Next token is: 25 Next lexeme is (
Next token is: 11 Next lexeme is sum
Next token is: 21 Next lexeme is +
Next token is: 10 Next lexeme is 47
Next token is: 26 Next lexeme is )
Next token is: 24 Next lexeme is /
Next token is: 11 Next lexeme is total
Next token is: -1 Next lexeme is EOF
```

Lexical Analysis

- **Example**

front.in

```
(sum + 47) / total
```

```
main(int argc, char** argv)
```

```
/*****************************************************/
/* main driver */
main() {

/* Open the input data file and process its contents */
  if ((in_fp = fopen("front.in", "r")) == NULL)
    printf("ERROR - cannot open front.in \n");
  else {
    getChar();
    do {
      lex();
    } while (nextToken != EOF);
  }
}
```

Lexical Analysis

- **Example**

front.in

```
(sum + 47) / total
```

```
charClass = LETTER
lexeme[100] = "(\0"
nextChar = 's'
lexLen = 1
token =
nextToken = LEFT_PAREN
```

getChar()
- nextChar = '('
- isalpha(nextChar) // F
- isdigit(nextChar) // F
- charClass = UNKNOWN

lex()
- lexLen = 0
- getNonBlank()
- switch(charClass) // UNKNOWN
  - **lookup**(nextChar)
  - **getChar**()
- **printf**(nextToken, lexeme)

lookup(char ch)
- switch(ch) // '('
  - **addChar**()
  - nextToken = LEFT_PAREN

addChar()
- lexeme[lexLen++] = nextChar
- lexeme[lexLen] = 0 // NULL

getChar()
- nextChar = 's'
- isalpha(nextChar) // T
- charClass = LETTER

Lexical Analysis

- **Example**

front.in

```
(sum + 47) / total
```

```
charClass = LETTER
lexeme[100] = "sum\0"
nextChar = 'm'
lexLen = 1
token =
nextToken = LEFT_PAREN
```

```
lex()
  - lexLen = 0
  - getNonBlank()
  - switch(charClass) // LETTER
    - addChar()
    - getChar()
    - while (charClass == LETTER ||
             charClass == DIGIT)
        - addChar()
        - getChar()
  - printf(nextToken, lexeme)
```

```
addChar()
  - lexeme[lexLen++] = nextChar
  - lexeme[lexLen] = 0; // NULL

getChar()
  - nextChar = ' '
  - isalpha(nextChar) // F
  - isdigit(nextChar) // F
  - charClass = UNKNOWN

getChar()
  - nextChar = '+'
  - isalpha(nextChar) // F
  - isdigit(nextChar) // F
  - charClass = UNKNOWN
```

The Parsing Problem (파싱 문제)

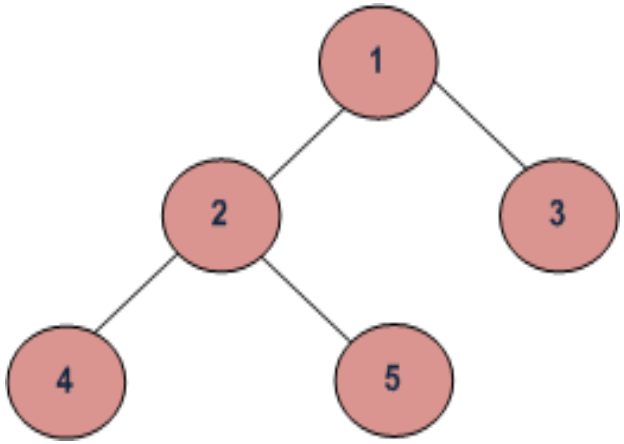
- **Parsing (파싱)**
  - The process of analyzing a sequence of tokens to determine whether it conforms to the syntax (grammar) of a programming language

- **Goals of the parser, given an input program:**
  - Find all syntax errors;
    - For each, produce an appropriate diagnostic message and recover quickly
  - Produce the parse tree, or at least a trace of the parse tree, for the program

The Parsing Problem

- **Two categories of parsers**
  - **Top down (하향식)**: produce the parse tree, beginning at the root
    - Order is that of a leftmost derivation
    - Traces or builds the parse tree in preorder (전위순회)
  - **Bottom up (상향식)**: produce the parse tree, beginning at the leaves
    - Order is that of the reverse of a rightmost derivation

Tree Traversals (Recap)



- **Depth First Traversals (or Searches):**
  - In-order (Left, Root, Right): 4 2 5 1 3
  - Pre-order (Root, Left, Right): 1 2 4 5 3
  - Post-order (Left, Right, Root): 4 5 2 3 1

The Parsing Problem

- **Notational conventions**
  - **Terminal symbols**: lowercase letters at the beginning of the alphabet (`a`, `b`, ...)
  - **Nonterminal symbols**: uppercase letters at the beginning of the alphabet (`A`, `B`, ...)
  - **Terminals and non-terminals**: uppercase letters at the end of the alphabet (`W`, `X`, `Y`, `Z`)
  - **Strings of terminals**: lowercase letters at the end of alphabet (`w`, `x`, `y`, `z`)
  - **Mixed strings (terminals and/or non-terminals)**: lowercase Greek letters ($\alpha, \beta, \gamma, \delta$)

The Parsing Problem

- **Top-down Parsers**
  - Given a sentential form(문장형태), **xAα**, the parser must choose the correct **A**-rule to get the next sentential form in the leftmost derivation, using only the first token produced by **A**
  - Example: **xAα**

    - A → bB, A → cBb, A → a

- **The most common top-down parsing algorithms:**
  - Recursive descent: a coded implementation
  - LL parsers: table driven implementation

The Parsing Problem

- **Example: Leftmost Derivation**

```
<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <term>
       | <term>
<term> → <term> * <factor>
       | <factor>
<factor> → ( <expr> )
         | <id>
```

Example: A = B + C * A

```
<assign> ⇒ <id> = <expr>
         ⇒ A = <expr>
         ⇒ A = <expr> + <term>
         ⇒ A = <term> + <term>
         ⇒ A = <factor> + <term>
         ⇒ A = <id> + <term>
         ⇒ A = B + <term>
         ⇒ A = B + <term> * <factor>
         ⇒ A = B + <factor> * <factor>
         ⇒ A = B + <id> * <factor>
         ⇒ A = B + C * <factor>
         ⇒ A = B + C * <id>
         ⇒ A = B + C * A
```

The Parsing Problem

- **Top-down Parsers**
  - Given a sentential form, **xAα**, the parser must choose the correct **A**-rule to get the next sentential form in the leftmost derivation, using only the first token produced by **A**
  - Example: **xAα**

    - `A → bB, A → cBb, A → a`

- **The most common top-down parsing algorithms:**
  - Recursive descent: a coded implementation
  - LL parsers: table-driven implementation

The Parsing Problem

- **Bottom-up parsers**
  - Given a right sentential form, α, determine what substring of α is the right-hand side of the rule in the grammar that must be ***reduced*** to produce the previous sentential form in the right derivation
  - The most common bottom-up parsing algorithms are in the LR family
  - Example: $S \rightarrow aAc, A \rightarrow aA \mid b$

$$S \Rightarrow aAc \Rightarrow aaAc \Rightarrow \boldsymbol{aabc}$$

The Parsing Problem

- **Example: Rightmost Derivation**

```
<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <term>
       | <term>
<term> → <term> * <factor>
       | <factor>
<factor> → ( <expr> )
         | <id>
```

Example: A = B + C * A

```
<assign> ⇒ <id> = <expr>
         ⇒ <id> = <expr> + <term>
         ⇒ <id> = <expr> + <term> * <factror>
         ⇒ <id> = <expr> + <term> * <id>
         ⇒ <id> = <expr> + <term> * A
         ⇒ <id> = <expr> + <factor> * A
         ⇒ <id> = <expr> + <id> * A
         ⇒ <id> = <expr> + C * A
         ⇒ <id> = <term> + C * A
         ⇒ <id> = <factor> + C * A
         ⇒ <id> = <id> + C * A
         ⇒ <id> = B + C * A
         ⇒ A = B + C * A
```

The Parsing Problem

- **Bottom-up parsers**
  - Given a right sentential form, α, determine what substring of α is the right-hand side of the rule in the grammar that must be ***reduced*** to produce the previous sentential form in the right derivation ***(Handle)***
  - The most common bottom-up parsing algorithms are in the LR family
  - Example: $S \rightarrow aAc, A \rightarrow aA \mid b$

$$S \Rightarrow aAc \Rightarrow aaAc \Rightarrow \boldsymbol{aabc}$$

A ? ? b

```
LHS → RHS
  S → aAc
  A → aA
  A → b
```

The Parsing Problem

- **The Complexity of Parsing**
  - Parsers that work for any unambiguous grammar are complex and inefficient (O(n^3), where n is the length of the input)
  - Compilers use parsers that only work for a subset of all unambiguous grammars, but do it in linear time (O(n), where n is the length of the input)

Recursive-Descent Parsing

- **A collection of subprograms**
  - Recursive
  - Produces a parse tree in top-down order
- **This recursion is a reflection of the nature of programming languages, which include several different kinds of nested structures.**
- **There is a subprogram for each nonterminal in the grammar, which can parse sentences that can be generated by that nonterminal**
- **EBNF is ideally suited for being the basis for a recursive-descent parser, because EBNF minimizes the number of nonterminals**

Recursive-Descent Parsing

- **A grammar for simple expressions:**

```
<expr> → <term> {(+|-) <term>}
<term> → <factor> {(*|/) <factor>}
<factor> → id | int_constant | ( <expr> )
```

Recursive-Descent Parsing

- **Assume we have a lexical analyzer named `lex`, which puts the next token code in `nextToken`**

- **The coding process when there is only one RHS:**
  - For each terminal symbol in the RHS, compare it with the next input token; if they match, continue, else there is an *error*
  - For each nonterminal symbol in the RHS, call its associated parsing subprogram

Recursive-Descent Parsing

```
/* expr
   Parses strings in the language generated by the rule:
   <expr> -> <term> {(+ | -) <term>}
   */
void  expr() {
  printf("Enter <expr>\n");

/* Parse the first term */
  term();

/* As long as the next token is + or -, get
   the next token and parse the next term */
  while  (nextToken == ADD_OP || nextToken == SUB_OP) {
    lex();
    term();
  }
  printf("Exit <expr>\n");
}  /* End of function expr */
```

Recursive-Descent Parsing

```
/* term
   Parses strings in the language generated by the rule:
   <term> -> <factor> {(* | /) <factor>)
   */
void  term() {
  printf("Enter <term>\n");

/* Parse the first factor */
  factor();

/* As long as the next token is * or /, get the
   next token and parse the next factor */
  while  (nextToken == MULT_OP || nextToken == DIV_OP) {
    lex();
    factor();
  }
  printf("Exit <term>\n");
}  /* End of function term */
```

Recursive-Descent Parsing

- **This particular routine does not detect errors**
- **Convention: Every parsing routine leaves the next token in `nextToken`**

  - Example: `sum + 37`

```
<expr> => <term> + <term>
       => <factor> + <term>
       => id + <term>
```

```
expr()
  - term()
    - factor()
       - id
       - nextToken = '+'
    - nextToken ?= '*' or '/' // F
    - factor() // F
  - nextToken ?= '+' or '-'
```

Recursive-Descent Parsing

- **A nonterminal that has more than one RHS requires an initial process to determine which RHS it is to parse**
  - The correct RHS is chosen on the basis of the next token of input (the lookahead)
  - The next token is compared with the first token that can be generated by each RHS until a match is found
  - If no match is found, it is a syntax error

Recursive-Descent Parsing

```
/* factor
   Parses strings in the language generated by the rule:
   <factor> -> id | int_constant | ( <expr >)
   */
void factor() {
  printf("Enter <factor>\n");

/* Determine which RHS */
  if (nextToken == IDENT || nextToken == INT_LIT)
    /* Get the next token */
        lex();

/* If the RHS is ( <expr>), call lex to pass over the
   left parenthesis, call expr, and check for the right
   parenthesis */
  else  {
    if  (nextToken == LEFT_PAREN) {
      lex();
```

nextToken = ?

Recursive-Descent Parsing

```
/* If the RHS is ( <expr>), call lex to pass over the
   left parenthesis, call expr, and check for the right
   parenthesis */
  else  {
    if  (nextToken == LEFT_PAREN) {
      lex();
      expr();
      if (nextToken == RIGHT_PAREN)
        lex();
      else
        error();
    }  /* End of if (nextToken == ... */

/* It was not an id, an integer literal, or a left
   parenthesis */
    else
      error();
  }  /* End of else */

  printf("Exit <factor>\n");;
}  /* End of function factor */
```

Recursive-Descent Parsing

- **Example (pp. 184-185): (sum + 47) / total**

```
Next token is: 25 Next lexeme is (
Enter <expr>
Enter <term> // (1)
Enter <factor> // (2)
Next token is: 11 Next lexeme is sum
Enter <expr> // (3)
Enter <term>
Enter <factor>
Next token is: 21 Next lexeme is +
Exit <factor>; Exit <term>
Next token is: 10 Next lexeme is 47
Enter <term>
Enter <factor>
Next token is: 26 Next lexeme is )
Exit <factor>; Exit <term>; Exit <expr>
Next token is: 24 Next lexeme is /
Exit <factor>
Next token is: 11 Next lexeme is total
Enter <factor>
Next token is: -1 Next lexeme is EOF
Exit <factor> Exit <term> Exit <expr>
```

```
<expr> => <term> // (1)
       => <factor> / <factor> // (2)
       => (<expr>) / <factor> // (3)
       => (<term> + <term>) / <factor>
       => (<factor> + <term>) / <factor>
       => (id + <term>) / <factor>
       => (id + <factor>) / <factor>
       => (id + int_const.) / <factor>
       => (id + int_const.) / id
       => (sum + 47) / total
```

Recursive-Descent Parsing

- **Example (pp. 184-185): (sum + 47) / total**

```
Next token is: 25 Next lexeme is (
Enter <expr>
Enter <term> // (1)
Enter <factor> // (2)
Next token is: 11 Next lexeme is sum
Enter <expr> // (3)
Enter <term>
Enter <factor>
Next token is: 21 Next lexeme is +
Exit <factor>; Exit <term>
Next token is: 10 Next lexeme is 47
Enter <term>
Enter <factor>
Next token is: 26 Next lexeme is )
Exit <factor>; Exit <term>; Exit <expr>
Next token is: 24 Next lexeme is /
Exit <factor>
Next token is: 11 Next lexeme is total
Enter <factor>
Next token is: -1 Next lexeme is EOF
Exit <factor> Exit <term> Exit <expr>
```

```
<expr> => <term> // (1)
       => <factor> / <factor> // (2)
       => (<expr>) / <factor> // (3)
       => (<term> + <term>) / <factor>
       => (<factor> + <term>) / <factor>
       => (id + <term>) / <factor>
       => (id + <factor>) / <factor>
       => (id + int_const.) / <factor>
       => (id + int_const.) / id
       => (sum + 47) / total
```

```
/* Determine which RHS */
  if (nextToken == IDENT || nextToken == INT_LIT)
    /* Get the next token */
      lex();
```

```
expr()
 ...
/* As long as the next token is + or -, get
   the next token and parse the next term */
  while  (nextToken == ADD_OP || nextToken == SUB_OP) {
    lex();
    term();
  }
```

Recursive-Descent Parsing

- **Example (pp. 184-185): (sum + 47) / total**

```
Next token is: 25 Next lexeme is (
Enter <expr>
Enter <term> // (1)
Enter <factor> // (2)
Next token is: 11 Next lexeme is sum
Enter <expr> // (3)
Enter <term>
Enter <factor>
Next token is: 21 Next lexeme is +
Exit <factor>; Exit <term>
Next token is: 10 Next lexeme is 47
Enter <term>
Enter <factor>
Next token is: 26 Next lexeme is )
Exit <factor>; Exit <term>; Exit <expr>
Next token is: 24 Next lexeme is /
Exit <factor>  // go to <term> // (1)
Next token is: 11 Next lexeme is total
Enter <factor>
Next token is: -1 Next lexeme is EOF
Exit <factor> Exit <term> Exit <expr>
```

```
factor()
 ...
/* If the RHS is ( <expr>), call lex to pass over the
   left parenthesis, call expr, and check for the right
   parenthesis */
 else  {
   if  (nextToken == LEFT_PAREN) {
     lex();
     expr();
     if (nextToken == RIGHT_PAREN)
       lex();
     else
       error();
   }  /* End of if (nextToken == ... */
```

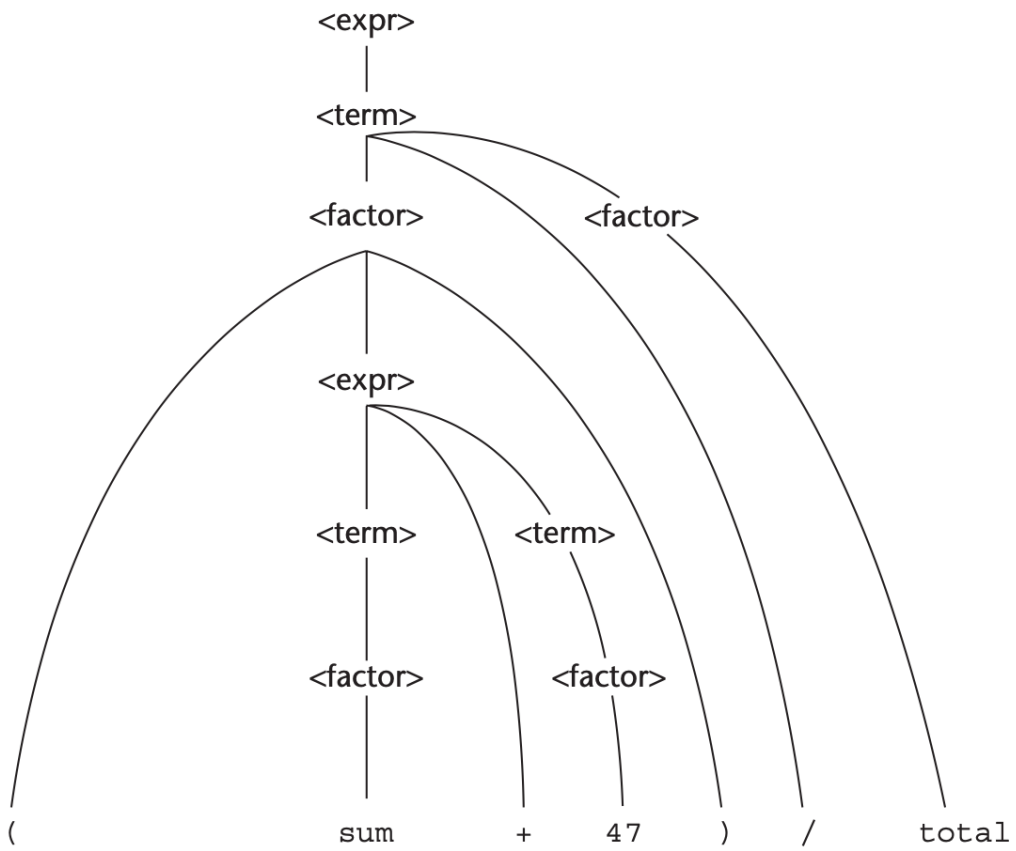
```
// go to <factor> // (2)

term()
 ...
/* As long as the next token is * or /, get the
   next token and parse the next factor */
 while  (nextToken == MULT_OP || nextToken == DIV_OP) {
   lex();
   factor();
 }
 printf("Exit <term>\n");
}  /* End of function term */
```

Recursive-Descent Parsing

- **Example (pp. 184-185)**

```
(sum + 47) / total
```



Recursive-Descent Parsing

- **LL parsing**
  - A top-down parsing technique that scans the input from **L**eft to right and constructs a **L**eftmost derivation, choosing productions based on lookahead tokens.
  - **LL Grammar**
    - A context-free grammar that can be parsed from left to right while constructing a leftmost derivation.
  - Example (LL(1) Parser)

$$S \rightarrow aAB$$
$$A \rightarrow bA \mid \epsilon$$
$$B \rightarrow c$$

Nonterminal	a	b	c
S	$S \rightarrow aAB$	error	error
A	error	$A \rightarrow bA$	$A \rightarrow \epsilon$
B	error	error	$B \rightarrow c$

Recursive-Descent Parsing

- **The LL Grammar Class (LL문법 유형)**
  - The Left Recursion Problem (좌-순환 문제)
    - If a grammar has left recursion, either direct or indirect, it cannot be the basis for a top-down parser
      - A grammar can be modified to remove direct left recursion as follows:

    - For each nonterminal, A,
      - Group the A-rules as

        A → Aα1 | ... | Aαm | β1 |β2 | ... | βn, where none of the β's begins with A
      - Replace the original A-rules with

        A → β1A' | β2A' | ... | βnA'

        A'→ α1A' | α2A' | ... | αmA' | ε

Recursive-Descent Parsing

- **The LL Grammar Class (LL문법 유형)**
  - The Left Recursion Problem (좌-순환 문제)

EXAMPLE

E → TA
A → +TA | ε
T → FB
B → *FB | ε
F → (E) | id

NT	first	Follow
E	(, id	$)
A	+, ε	$)
T	(, id	+ $)
B	*, ε	+ $)
F	(, id	* + $)

T / NT	+	*	(	)	id	$
E			E→TA		E→TA	
A	A→+TA			A→ε		A→ε
T			T→FB		T→FB	
B	B→ε	B→*FB		B→ε		B→ε
F			F→(E)		F→id	

Recursive-Descent Parsing

- **The LL Grammar Class (LL문법 유형)**
  - The Left Recursion Problem (좌-순환 문제)
    - If a grammar has left recursion, either direct or indirect, it cannot be the basis for a top-down parser
      - A grammar can be modified to remove direct left recursion as follows:

    - For each nonterminal, A,
      - Group the A-rules as

        A → Aα1 | ... | Aαm | β1 |β2 | ... | βn, where none of the β's begins with A
      - Replace the original A-rules with

        A → β1A' | β2A' | ... | βnA'

        A'→ α1A' | α2A' | ... | αmA' | ε

Recursive-Descent Parsing

- **The LL Grammar Class**
  - The other characteristic of grammars that disallows top-down parsing is the lack of pairwise disjointness (집합쌍 공통)
    - The inability to determine the correct RHS on the basis of one token of lookahead
    - Def: `FIRST(α) = {a | α =>* aβ}`
      (If `α =>* ε`, `ε` is in `FIRST(α)`)

Recursive-Descent Parsing

- **The LL Grammar Class**
  - Pairwise Disjointness Test (집합쌍 공통 테스트):
    - For each nonterminal, A, in the grammar that has more than one RHS, for each pair of rules, $A \rightarrow \alpha_i$ and $A \rightarrow \alpha_j$, it must be true that

      $$\texttt{FIRST}(\alpha_i) \cap \texttt{FIRST}(\alpha_j) = \emptyset$$

  - Example:

    A → a | bB | cAb

    A → a | aB

Recursive-Descent Parsing

- **The LL Grammar Class**
  - Left factoring (좌추출) can resolve the problem

Replace

```
<variable> → identifier | identifier [<expression>]
```

with

```
<variable> → identifier <new>
<new> → ϵ | [<expression>]
```

or

```
<variable> → identifier [[<expression>]]
```

(the outer brackets are meta-symbols of EBNF)

Bottom-Up Parsing

- **The parsing problem is finding the correct RHS in a right-sentential form to reduce to get the previous right-sentential form in the derivation**

- **Example**
  - $E \rightarrow E + T \mid T$
  - $T \rightarrow T * F \mid F$
  - $F \rightarrow (E) \mid id$

```
input: id + id * id
```

```
E => E + T
  => E + T * F
  => E + T * id
  => E + F * id
  => E + id * id
  => T + id * id
  => F + id * id
  => id + id * id
```

Bottom-Up Parsing

- **The parsing problem is finding the correct RHS in a right-sentential form to reduce to get the previous right-sentential form in the derivation**

- **Example**
  - $E \rightarrow E + T \mid T$
  - $T \rightarrow T * F \mid F$
  - $F \rightarrow (E) \mid id$

```
input: id + id * id
```

```
E => E + T
  => E + T * F
  => E + T * id
  => E + F * id
  => E + id * id
  => T + id * id
  => F + id * id
  => id + id * id
```

```
=> E + T * F
=> E + T * id
```

handle!

```
=> E * id
=> E + T * id
```

```
=> E + E * id
=> E + T * id
```

Bottom-Up Parsing

- **Intuition about handles:**
  - Def: β is the **handle (핸들)** of the right sentential form $\gamma = \alpha\beta w$ *iff*

    $S =>^*_{rm} \alpha A w =>_{rm} \alpha \boldsymbol{\beta} w$

  - Def: β is a **phrase (어구)** of the right sentential form γ *iff*

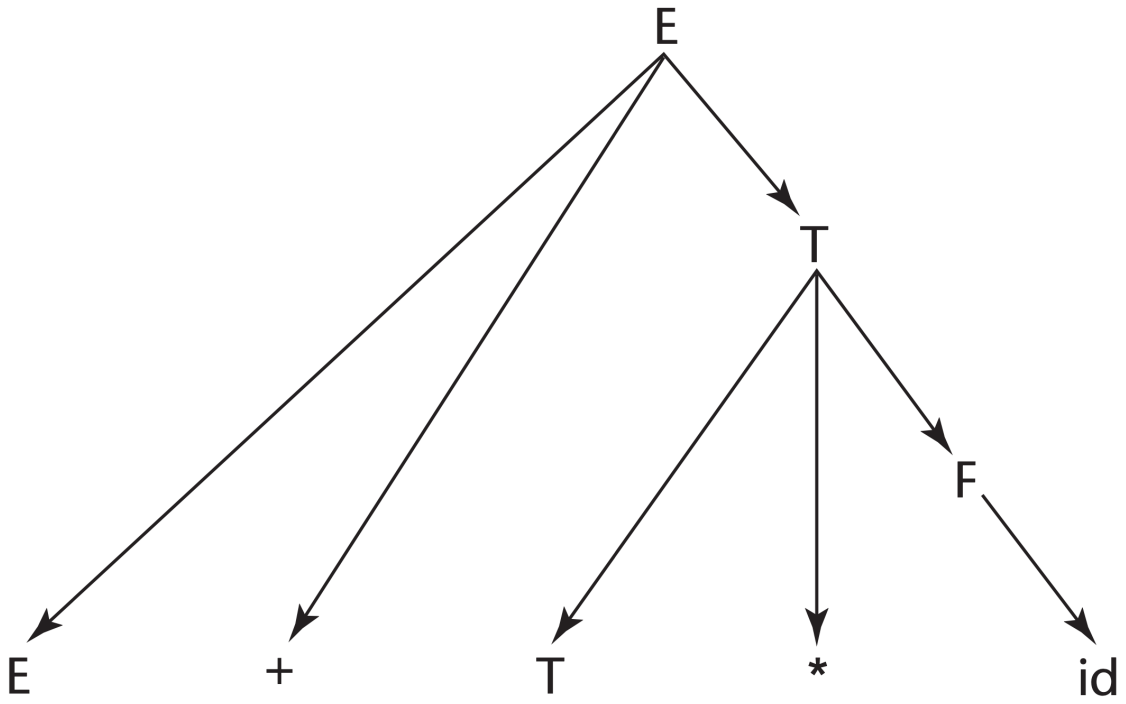
    $S =>^* \gamma = \alpha_1 A \alpha_2 =>^+ \alpha_1 \boldsymbol{\beta} \alpha_2$

  - Def: β is a **simple phrase (단순어구)** of the right sentential form γ *iff*

    $S =>^* \gamma = \alpha_1 A \alpha_2 => \alpha_1 \boldsymbol{\beta} \alpha_2$

Bottom-Up Parsing

- **Example**
  - E + T * id



Bottom-Up Parsing

- **Intuition about handles (cont.):**
  - The handle of a right sentential form is its leftmost simple phrase
  - Given a parse tree, it is now easy to find the handle
  - Parsing can be thought of as handle pruning

Bottom-Up Parsing

- **Shift-Reduce Algorithms**
  - **Shift** is the action of moving the next token to the top of the parse stack
  - **Reduce** is the action of replacing the handle on the top of the parse stack with its corresponding LHS

Bottom-Up Parsing

- **Advantages of LR parsers:**
  - They will work for nearly all grammars that describe programming languages.
  - They work on a larger class of grammars than other bottom-up algorithms but are as efficient as any other bottom-up parser.
  - They can detect syntax errors as soon as it is possible.
  - The LR class of grammars is a superset of the class parsable by LL parsers.

Bottom-Up Parsing

- **Advantages of LR parsers:**
  - They will work for nearly all grammars that describe programming languages.
  - They work on a larger class of grammars than other bottom-up algorithms but are as efficient as any other bottom-up parser.
  - They can detect syntax errors as soon as it is possible.
  - The LR class of grammars is a superset of the class parsable by LL parsers.

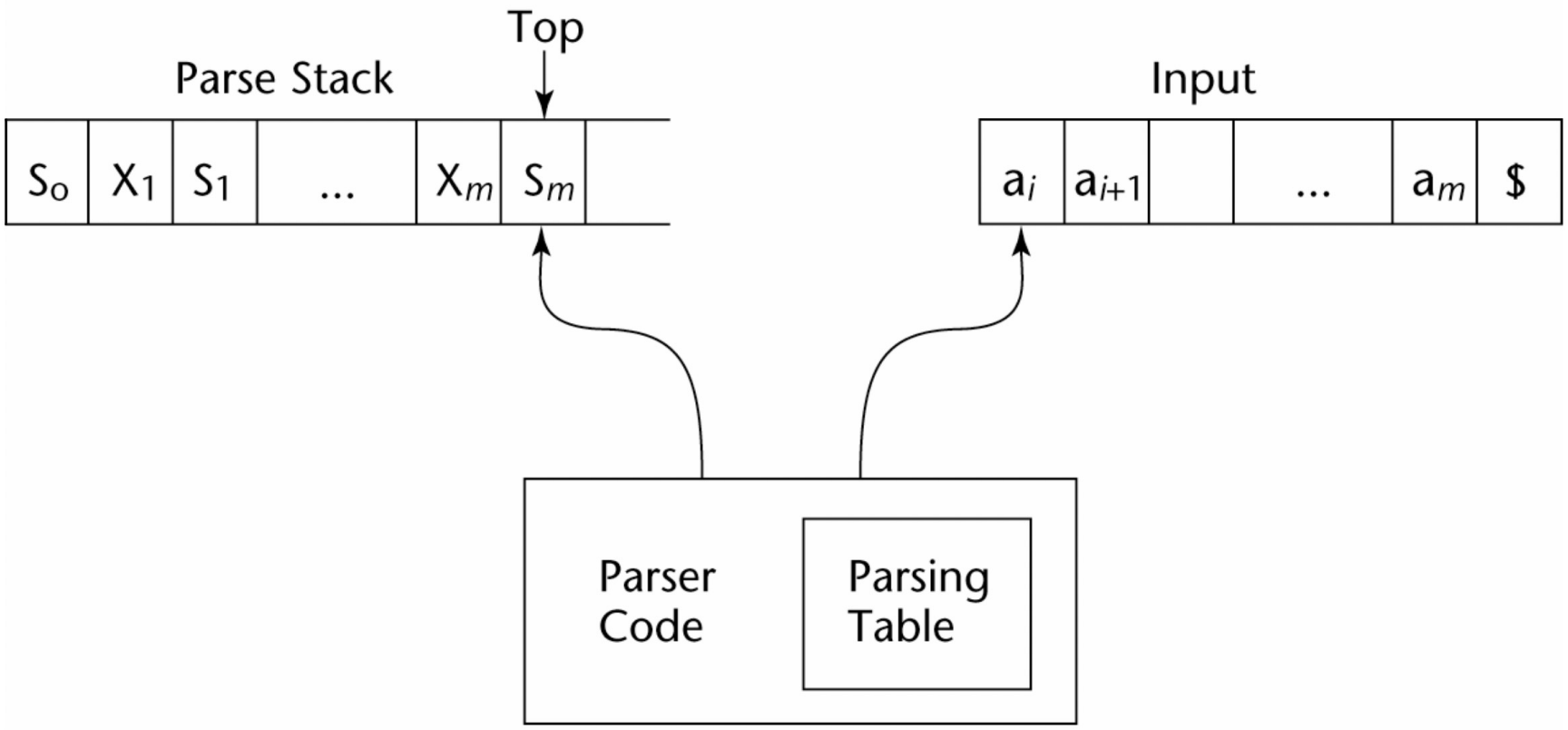
Bottom-Up Parsing

- **LR parsers must be constructed with a tool**
  - A parser table can be generated from a given grammar with a tool, e.g., yacc or bison

- **Knuth's insight: A bottom-up parser could use the entire history of the parse, up to the current point, to make parsing decisions**
  - There are only a finite and relatively small number of different parse situations that could have occurred, so the history could be stored in a parser state, on the parse stack

Bottom-Up Parsing

- **Structure of an LR Parser**



Bottom-Up Parsing

- **An LR configuration stores the state of an LR parser**

$$(S_0 X_1 S_1 X_2 S_2 \ldots X_m S_m, a_i a_{i+1} \ldots a_n \$)$$

Bottom-Up Parsing

- **LR parsers are table driven, where the table has two components, an ACTION table and a GOTO table**
  - The ACTION table specifies the action of the parser, given the parser state and the next token
    - Rows are state names; columns are terminals
  - The GOTO table specifies which state to put on top of the parse stack **after** a reduction action is done
    - Rows are state names; columns are nonterminals

	Action						Goto		
State	id	+	*	(	)	$	E	T	F
0	S5		S4				1	2	3
1		S6				accept			

Bottom-Up Parsing

- **Initial configuration:** $(S_0, a_1 \ldots, a_n\$)$
- **Parser actions:**
  - **For a Shift**, the next symbol of input is pushed onto the stack, along with the state symbol that is part of the Shift specification in the Action table
  - **For a Reduce**, remove the handle from the stack, along with its state symbols. Push the LHS of the rule. Push the state symbol from the GOTO table, using the state symbol just below the new LHS in the stack and the LHS of the new rule as the row and column into the GOTO table

Bottom-Up Parsing

- **Parser actions (continued):**
  - For an Accept, the parse is complete, and no errors were found.
  - For an Error, the parser calls an error-handling routine.

```
1.E → E + T
2.E → T
3.T → T * F
4.T → F
5.F → (E)
6.F → id
```

Bottom-Up Parsing

- **LR Parsing Table**

	Action						Goto		
State	id	+	*	(	)	$	E	T	F
0	S5		S4				1	2	3
1		S6				accept			
2		R2	S7		R2	R2			
3		R4	R4		R4	R4			
4	S5			S4			8	2	3
5		R6	R6		R6	R6			
6	S5			S4				9	3
7	S5			S4					10
8		S6			S11				
9		R1	S7		R1	R1			
10		R3	R3		R3	R3			
11		R5	R5		R5	R5			

Bottom-Up Parsing

- **Example:** `id + id * id`

1. E → E + T
2. E → T
3. T → T * F
4. T → F
5. F → (E)
6. F → id

Stack	Input
0	id + id * id $
0id5	+ id * id $
0F3	+ id * id $
0T2	+ id * id $
0E1	+ id * id $
0E1+6	id * id $
0E1+6id5	* id $
0E1+6F3	* id $
0E1+6T9	* id $
0E1+6T9*7	id $
0E1+6T9*7id5	$
0E1+6T9*7F10	$
0E1+6T9	$
0E1	$

	Action						Goto		
State	id	+	*	(	)	$	E	T	F
0	S5		S4				1	2	3
1		S6				accept			
2		R2	S7		R2	R2			
3		R4	R4		R4	R4			
4	S5			S4			8	2	3
5		R6	R6		R6	R6			
6	S5			S4				9	3
7	S5			S4					10
8		S6			S11				
9		R1	S7		R1	R1			
10		R3	R3		R3	R3			
11		R5	R5		R5	R5			

Summary

- **Syntax analysis is a common part of language implementation**
- **A lexical analyzer is a pattern matcher that isolates small-scale parts of a program**
  - Detects syntax errors
  - Produces a parse tree
- **A recursive-descent parser is an LL parser**
  - EBNF
- **Parsing problem for bottom-up parsers: find the substring of current sentential form**
- **The LR family of shift-reduce parsers is the most common bottom-up parsing approach**

Q & A